CMSC132: Abstract Data Types

Chapter 5

By Naren Ayinala

May 2026

Contents

1 Abstract Data Types: Behavior Before Implementation **2**

1.1 Stack ADT (Last In, First Out) 2

1.2 Queue ADT (First In, First Out) 2

2 List, Set, and Map ADTs **2**

3 Specifying an ADT with a Java Interface **3**

4 Preconditions, Postconditions, and Invariants **3**

4.1 Implementation Choices and Complexity 3

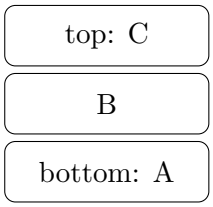
4.2 Practice 4

1 Abstract Data Types: Behavior Before Implementation

An **abstract data type (ADT)** describes a set of values and permitted operations independently of how they are stored. Think of a stack: you know that `push` adds to the top and `pop` removes the most recently added item. Whether the stack uses an array or linked nodes is a separate implementation decision. An ADT is a conceptual specification; a Java interface is one way to express it.

1.1 Stack ADT (Last In, First Out)

A stack is **LIFO**: the last item pushed is the first popped. Typical operations are `push`, `pop`, `peek`, `isEmpty`, and `size`. Applications include undo history, matching parentheses, and depth-first search.

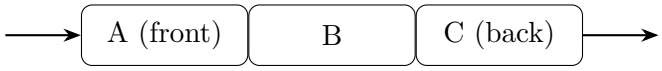


pop removes C

```
import java.util.*;
Deque<String> stack = new ArrayDeque<>();
stack.push("A");
stack.push("B");
stack.push("C");
System.out.println(stack.peek()); // C
System.out.println(stack.pop()); // C
System.out.println(stack.pop()); // B
```

1.2 Queue ADT (First In, First Out)

A queue is **FIFO**: the first element enqueued is the first removed. Typical operations are `offer/add`, `poll/remove`, and `peek`. Queues model printer jobs, customer service lines, and breadth-first search.




```
Queue<Integer> queue = new ArrayDeque<>();
queue.offer(10);
queue.offer(20);
queue.offer(30);
System.out.println(queue.poll()); // 10
System.out.println(queue.peek()); // 20
```

2 List, Set, and Map ADTs

A **list** is an ordered sequence with positions and usually allows duplicates. A **set** represents distinct elements without duplicate membership. A **map** associates unique keys with values. These are behaviors: `ArrayList` and `LinkedList` implement list behavior differently; `HashSet` and `TreeSet` implement set behavior differently.

ADT	Key property	Typical operations
List	Ordered sequence	get, add, remove
Set	Unique elements	add, contains, remove
Map	Unique keys	put, get, remove
Stack	LIFO	push, pop, peek
Queue	FIFO	offer, poll, peek

3 Specifying an ADT with a Java Interface

```
interface SimpleStack<T> {
    void push(T value);
    T pop();
    T peek();
    boolean isEmpty();
}
class ArrayStack<T> implements SimpleStack<T> {
    private final java.util.ArrayList<T> data =
        new java.util.ArrayList<>();
    public void push(T v) { data.add(v); }
    public T pop() {
        if (isEmpty()) throw new java.util.NoSuchElementException();
        return data.remove(data.size() - 1);
    }
    public T peek() {
        if (isEmpty()) throw new java.util.NoSuchElementException();
        return data.get(data.size() - 1);
    }
    public boolean isEmpty() { return data.isEmpty(); }
}
```

The interface does not say which data structure to use. The class uses an `ArrayList` internally. Another class could implement the same contract using a linked list. This separation makes code easier to test and change.

4 Preconditions, Postconditions, and Invariants

An ADT specification should say what happens for edge cases. **Precondition**: what must be true before an operation, such as a stack being nonempty before popping. **Postcondition**: what the operation guarantees afterward, such as the stack size decreasing by one. **Invariant**: what remains true across valid operations, such as a stack's elements preserving LIFO order.

4.1 Implementation Choices and Complexity

Array-backed lists provide $O(1)$ indexed access, but insertion near the beginning may shift $O(n)$ elements. Linked lists can insert at a known node in $O(1)$ but locating that node often costs $O(n)$. A hash-based set provides expected $O(1)$ membership testing with a suitable hash function; a balanced-tree set usually provides $O(\log n)$ membership testing and sorted iteration. The *ADT contract* stays the same while implementation performance changes.

4.2 Practice

1. Does "stack" mean "linked list"? No: stack is an ADT, linked list is a possible representation. **2.** Which ADT models browser back history? A stack. **3.** Which ADT models requests processed in arrival order? A queue. **4.** Is `HashMap` an ADT? No: it is a concrete implementation of the map ADT.